IT 360 Cloud Integration | Week 3 | Team Lab 2 (Canvas 3.3, 40 points)

Lab 2: Docker and a Jenkins CI/CD Pipeline

Bonneville Brewing: a Jenkins job anyone can click that builds the menu-board image, tests it, tags it with the build number and pushes it with a password nobody can see

---

Version 2, November 2026. The pipeline in this guide was run end to end on November 8, 2026, including a real push to Docker Hub (every build green, the token in none of the logs). Your build numbers, times and image digests will differ; the shape of the output will not.

> **Duration: about 90 minutes in total (75 in class plus 15 to get ready)**
>
> **Before class:** 15 minutes (Docker Desktop test, Docker Hub account and access token, GitHub account, lab folder from the class repository). Do this before class if you can.
>
> **In class:** 75 minutes of lab work in six parts (A 10, B 20, C 15, D 10, E 10, F 10). Part B is the longest because the Jenkins image builds once (3 to 6 minutes); do Part A on GitHub while it builds.
>
> **Instructor's timed run (November 8, 2026, real Docker Hub push):** Jenkins sign-in page up 9 seconds after start, build #1 finished in 20 seconds, build #2 in 15 seconds; your first build can take a few minutes while Docker downloads NGINX.
>
> **Allow extra time if:** Docker Desktop is not installed yet (15 to 30 minutes to install and restart) or your laptop cannot run Docker and you use a cloud VM instead.

The story

Bonneville Brewing shows its beer list on screens in four taprooms. The screens run a small web page packaged as a Docker image. Today one person builds that image on his laptop and pushes it to Docker Hub with his personal password, so nobody else can change the menu when he is away. The owner wants a **Jenkins job** anyone on staff can click. It must build the image, test it, tag it with the build number, and push it with a stored credential that **never appears in a log**.

What you will build

Part	What you do	Time
A. The code	Put the menu board (3 files) in a GitHub repository	10 min
B. Jenkins	Run Jenkins in Docker and store your Docker Hub token in it	20 min
C. The pipeline	Create the job, Build Now, read the console	15 min
D. Change and repeat	Add a beer, build again, watch the tags	10 min
E. Connectivity	Test every hop from browser to taproom screen	10 min
F. Submit and clean up	Screenshots to Canvas, then remove the containers	10 min
Total	Six parts in one lab session	about 75 min, plus 15 min before class

Team roles

Role	Owns
Jenkins administrator	Part B: runs Jenkins and creates the stored credential
Pipeline author	Parts A and C: the GitHub repository, the Jenkinsfile and the job
Registry owner	Part D: Docker Hub repository and the tags
Quality lead	Parts D and E: searches every log for the password and writes the one-line sign-off

Run Jenkins on **one** laptop and share that screen. Teammates follow along and take over their parts.

Before you start (15 minutes, ideally before class)

- ☐ **Docker Desktop** is installed and running (whale icon in the menu bar on Mac, in the system tray on Windows). Test it in a terminal (Mac: Terminal; Windows: PowerShell): `docker run --rm hello-world` must print `Hello from Docker!`.
- ☐ **Docker Hub account** (free) at https://hub.docker.com. Write your username in the table below.
- ☐ **Docker Hub access token:** your avatar (top right) > Account settings > Personal access tokens > Generate new token. Description `jenkins-lab`, access **Read & Write**, Generate. Copy the token into your password manager now; Docker Hub shows it only once. Never paste it into a file, a chat or a screenshot.
- ☐ **GitHub account** at https://github.com.
- ☐ **The lab folder** from the class repository **https://github.com/isijackson/IT360-Fall-Labs-v2**: click the green **Code** button, then **Download ZIP**, and unzip it. The folder you need is `Week 3 labs/jenkins-docker-pipeline`, which contains the folders `jenkins` and `menu-board`. (Git users: `git clone https://github.com/isijackson/IT360-Fall-Labs-v2.git`.)
- ☐ No laptop that can run Docker? Use a Vocareum AWS, Azure or Google Cloud VM with Docker installed; the commands are the same.

Item	Value in this guide	Yours
Docker Hub username	`<user>`	
Docker Hub token	stored in Jenkins only	never write it here
GitHub repository	https://github.com/<github-user>/menu-board	
Jenkins	http://localhost:8080, admin / admin	same
Credential ID	`dockerhub-credentials` (exact spelling)	same

Where you run things

Place	How to open it	What you do there
Terminal (Mac) or **PowerShell** (Windows) on your laptop	Part B Step 1 shows how to open it **inside the `jenkins-docker-pipeline` folder**	Every `docker` command: build Jenkins, start it, run the image
GitHub in your web browser	https://github.com	Part A (create `menu-board`, upload 3 files, edit line 4) and Part D Step 3 (add a beer)

Place	How to open it	What you do there
Jenkins in your web browser	http://localhost:8080 (sign in as admin / admin)	Part B Step 5 (the credential), Part C (the job, Build Now, Console Output), Part D Step 1
Docker Hub in your web browser	https://hub.docker.com	Before class (the access token) and Part D Step 2 (the tags)
Cloud Shell	Not used in this lab	Nothing: Lab 2 runs on your laptop's Docker

Every command block below starts with an orange ▸ **Run in:** line that tells you exactly where to run it.

> **How to read this guide**
>
> **Click:** numbered steps in a web page (GitHub, Jenkins, Docker Hub). Screenshots show the same clicks with red numbers.
>
> **Type:** commands for your terminal. Each command is one line, so it works the same in Mac Terminal and Windows PowerShell.
>
> **You should see:** green boxes with the real output from the November 8 run.
>
> **If not:** yellow boxes with the fix.
>
> Replace anything in angle brackets, like `<user>`, including the brackets.

Part A. Put the code on GitHub (Pipeline author, 10 min)

Goal: a GitHub repository Jenkins can read. It holds three files: index.html (the menu), Dockerfile (how to build the image) and Jenkinsfile (the pipeline).

1. On GitHub, click **+** (top right) > **New repository**. Name: menu-board. Choose **Public**. Click **Create repository**.
2. On the empty repository page, click **uploading an existing file**.
3. Drag the three files from the menu-board folder of the class repository download into the page: index.html, Dockerfile, Jenkinsfile. Click **Commit changes**. (Upload only these three; your own repository must have them at the top level, so do not fork or upload the whole class repository.)
4. Click Jenkinsfile, then the pencil icon (Edit).
5. On line 4, replace YOUR_DOCKERHUB_USER with your Docker Hub username. Keep the quotes and the /menu-board part.
6. Click **Commit changes** twice.

> **You should see**
>
> Three files in the repository, and line 4 of the Jenkinsfile reading (with your username):
>
> ```
> IMAGE_NAME = "${env.IMAGE_NAME ?: '<user>/menu-board'}"
> ```
>
> The Jenkinsfile names the credential by its ID (dockerhub-credentials) and never contains a password. Anyone can read it safely.

The pipeline has four stages. Read them once now so the console output in Part C makes sense:

Stage	What it does
Checkout	Downloads your repository
Build	docker build: makes the image and tags it with the build number, for example <user>/menu-board:1
Smoke test	Starts the new image as a temporary container and checks the page answers with HTTP 200. If it does not, the pipeline stops and nothing is pushed.
Push	Logs in to Docker Hub with the stored credential and pushes two tags: the build number and latest

Part B. Run Jenkins and store the credential (Jenkins administrator, 20 min)

Goal: Jenkins running at http://localhost:8080, able to use Docker, holding your Docker Hub token as a credential.

Step 1. Open a terminal in the lab folder

Mac: open Terminal, type `cd` (with a space), drag the `jenkins-docker-pipeline` folder into the window, press Enter. Windows: in File Explorer open the `jenkins-docker-pipeline` folder, click the address bar, type `powershell`, press Enter. Check you are in the right place:

▸ Run in: Terminal (Mac) or PowerShell (Windows) on your laptop, after moving into the `jenkins-docker-pipeline` folder as described above.

```
ls
```

> **You should see**
>
> `jenkins` and `menu-board` in the list (plus a few other files).

Step 2. Build the Jenkins image (one time, 3 to 6 minutes)

▸ Run in: Terminal (Mac) or PowerShell (Windows) on your laptop, inside the `jenkins-docker-pipeline` folder (the one that contains `jenkins` and `menu-board`).

```
docker build -t it360-jenkins jenkins
```

This image is the official Jenkins plus the Docker command and the Docker Pipeline plugin. It also skips the setup wizard and creates the sign-in **admin / admin** for you.

> **You should see (last lines)**
>
> `=> naming to docker.io/library/it360-jenkins`
> No red ERROR lines. A second run is much faster because Docker reuses the layers.

Step 3. Create the lab network and start Jenkins

▸ Run in: Terminal (Mac) or PowerShell (Windows) on your laptop, the same window and folder as Step 2. Paste one line at a time.

```
docker network create brewlab
docker run -d --name jenkins --network brewlab -e SMOKE_NET=brewlab -p 8080:8080 -v
/var/run/docker.sock:/var/run/docker.sock it360-jenkins
docker exec -u root jenkins chmod 666 /var/run/docker.sock
```

> **Verify Jenkins is running and can use Docker (same terminal)**
>
> ```
> docker ps --filter name=^jenkins$ --format "{{.Names}}  {{.Status}}  {{.Ports}}"
> docker exec jenkins docker version --format "docker CLI {{.Client.Version}}, server {{.Server.Version}}"
> ```
> You should see (versions may differ):
>
> ```
> jenkins  Up 20 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
> docker CLI 27.5.1, server 28.4.0
> ```
> If the second command says `permission denied`, run the `chmod` line again. If `docker ps` shows nothing, run `docker logs jenkins` and read the last lines.

Why these options matter

`-v /var/run/docker.sock:...` lets Jenkins use your laptop's Docker to build images.

Because of that, the smoke-test container Jenkins starts is a **neighbour** of Jenkins, not inside it. `--network brewlab` and `-e SMOKE_NET=brewlab` let Jenkins reach that neighbour by name. Leave them out and the smoke test fails with "connection refused".

The `chmod` line lets the Jenkins user talk to Docker. Without it builds fail with `permission denied ... docker.sock`.

Step 4. Sign in

1. Wait about a minute, then open http://localhost:8080 in your browser.
2. Sign in as **admin**, password **admin**. (This is a lab-only account on your laptop.)

You should see

The Jenkins dashboard: Welcome to Jenkins!, with **New Item** and **Manage Jenkins** on the left.

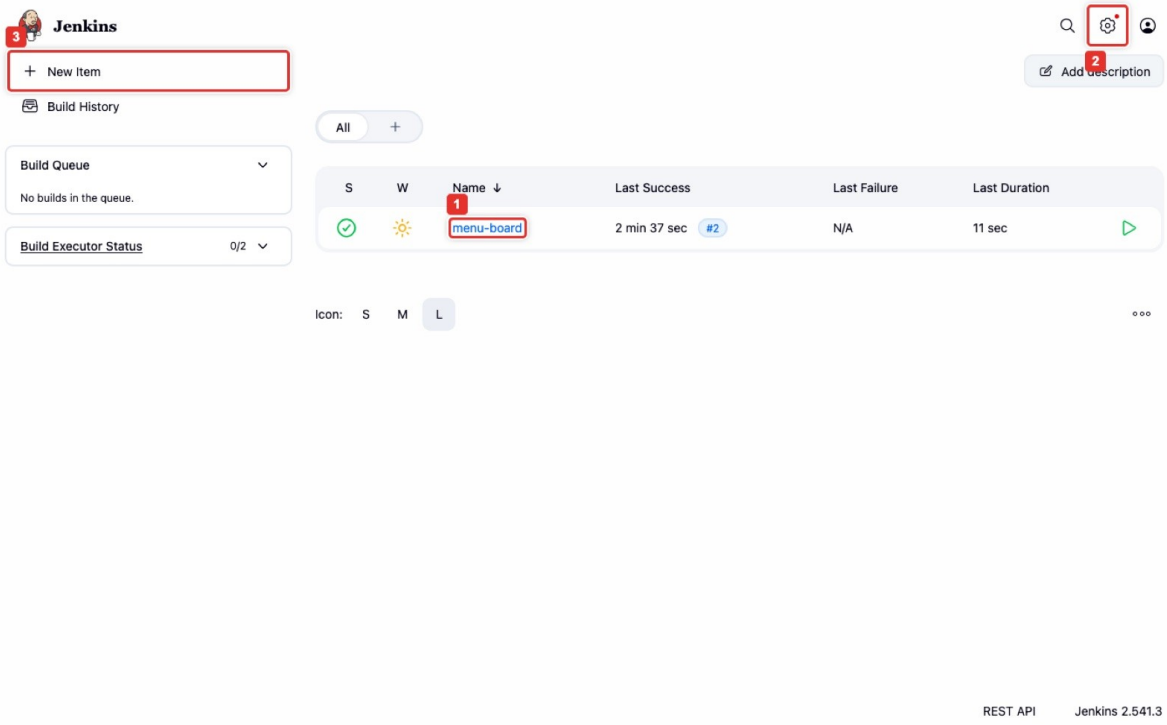


Figure 1. The Jenkins dashboard. Yours has no menu-board job yet; you create it in Part C.

Step 5. Store your Docker Hub token as a credential

1. Click **Manage Jenkins** > **Credentials**.
2. Under Stores scoped to Jenkins, click **System**, then **Global credentials (unrestricted)**.
3. Click **+ Add Credentials**.
4. Kind: **Username with password**. Scope: Global.
5. Username: your Docker Hub username. Password: your Docker Hub **access token** (not your Docker Hub password).
6. ID: `dockerhub-credentials` (exactly this; the Jenkinsfile looks for it). Description: `Docker Hub`.

7. Click **Create**.

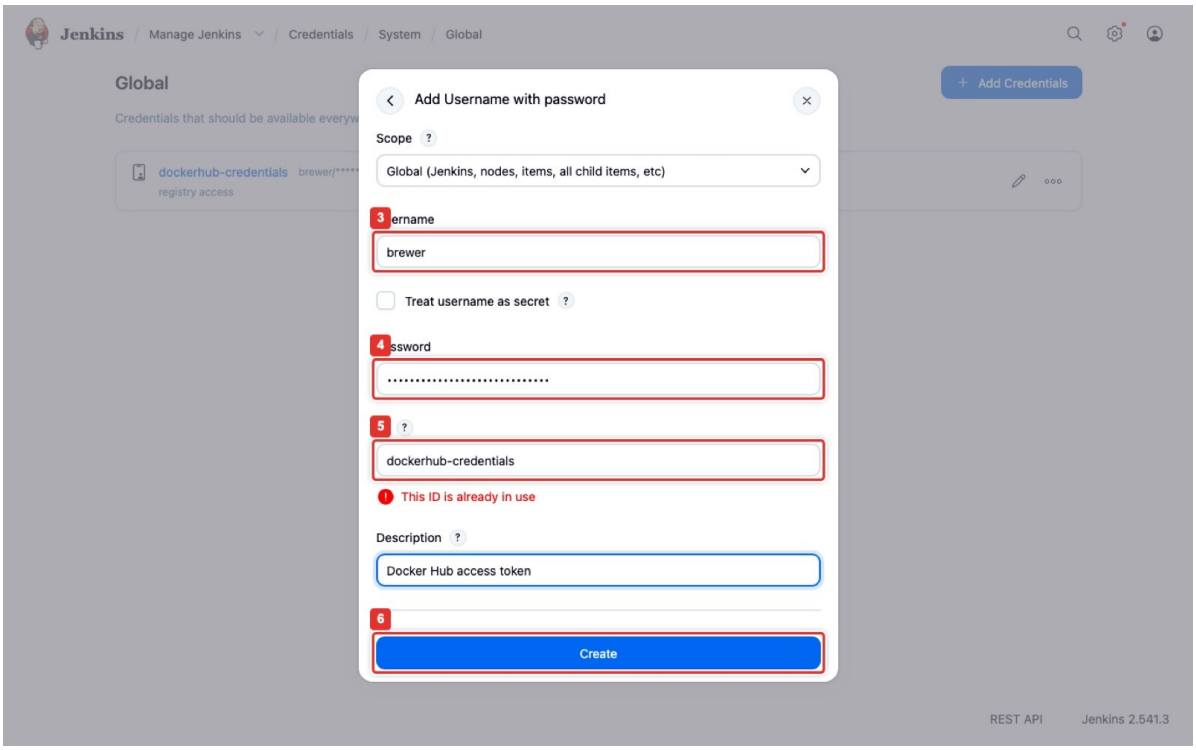


Figure 2. Add Credentials: Username with password, the token in Password, ID dockerhub-credentials.

You should see

The list shows **dockerhub-credentials** as <user>/******. The token is now stored encrypted, and Jenkins will never show it again, not even to you.

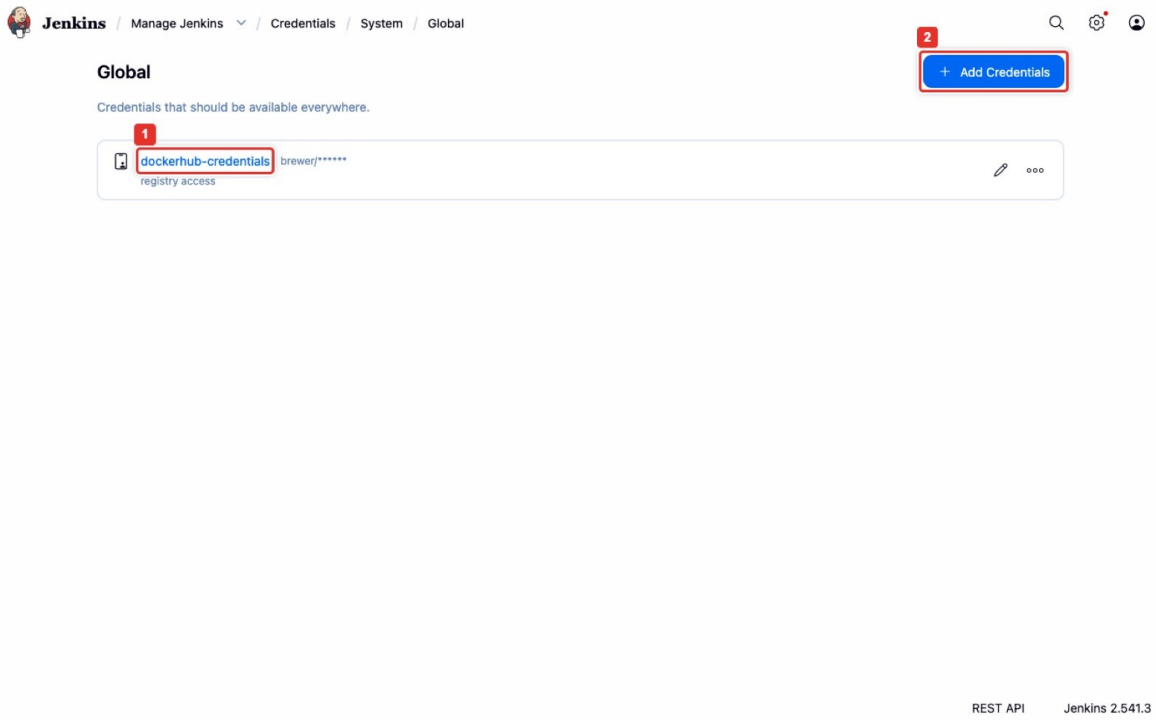


Figure 3. Global credentials: dockerhub-credentials listed with the secret hidden.

Checkpoint: where does the token live now, and who can read it?

Encrypted inside Jenkins. Only jobs that name the ID `dockerhub-credentials` can use it, and they never see it printed.

If not

port is already allocated on `docker run`: something else uses port 8080. Run `docker rm -f jenkins`, then the `docker run` line again with `-p 8090:8080`, and use http://localhost:8090 from now on.

Conflict. The container name "/jenkins" is already in use: an old Jenkins is there. `docker rm -f jenkins`, then run Step 3 again (skip `network create` if it says the network exists).

The page does not open: wait another minute; `docker logs jenkins` should end with `Jenkins is fully up and running`.

Part C. Create the pipeline and run it (Pipeline author, 15 min)

Goal: a job named menu-board that reads the Jenkinsfile from your GitHub repository, and a first green build.

1. On the dashboard, click **+ New Item**.
2. Name: menu-board. Choose **Pipeline**. Click **OK**.
3. Scroll to the **Pipeline** section. Definition: **Pipeline script from SCM**.
4. SCM: **Git**. Repository URL: https://github.com/<github-user>/menu-board.git. Leave Credentials as none (the repository is public).
5. Branch Specifier: change */master to */main.
6. Script Path: Jenkinsfile (already filled in).
7. Click **Save**.
8. Click **Build Now**. Build **#1** appears under Build History. Click **#1**, then **Console Output**.

> **Verify the job (Jenkins in your browser, after step 7)**
>
> The page heading reads **menu-board** with **Build Now** and **Configure** on the left. Click **Configure** and scroll to Pipeline: it still shows **Pipeline script from SCM**, your repository URL, */main and Jenkinsfile. If any value is empty, fix it and Save again before Build Now.

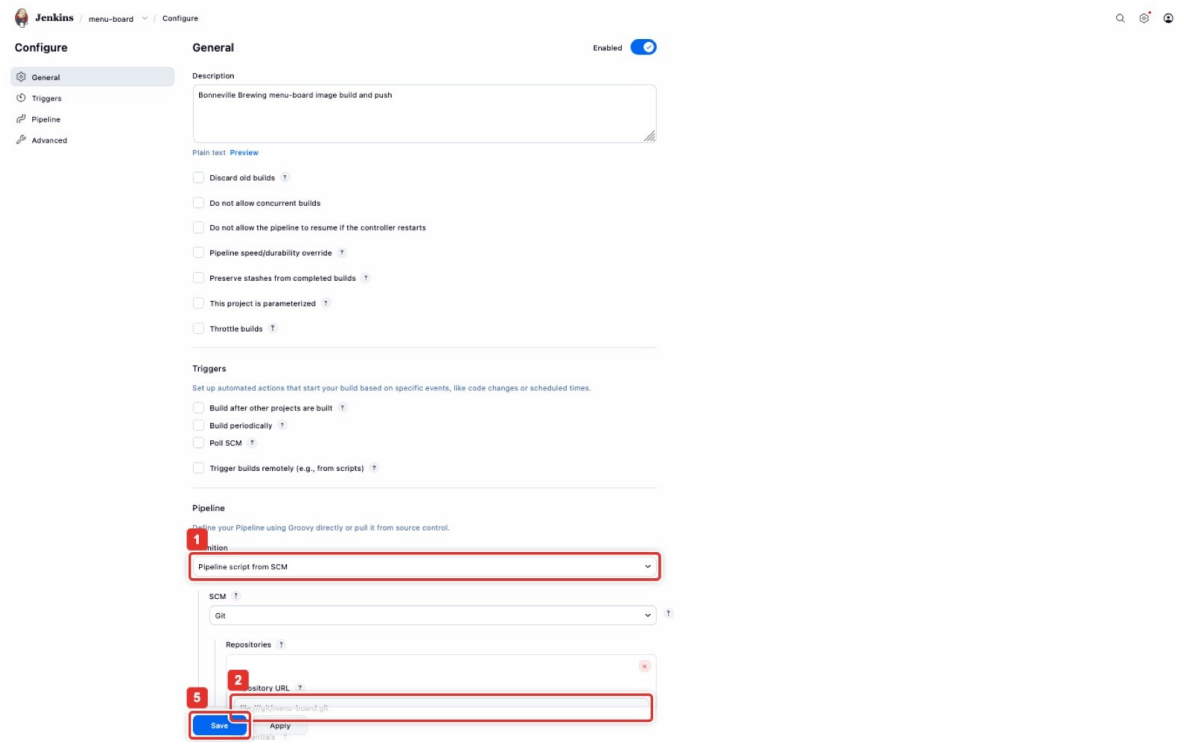


*Figure 4. Configure > Pipeline: Pipeline script from SCM, Git, your repository URL, branch */main, Save.*

You should see in Console Output (the important lines)

```
Checking out Revision ... (refs/remotes/origin/main)
+ docker build -t <user>/menu-board:1 .
HTTP 200
$ docker login -u <user> -p ******** https://index.docker.io/v1/
Login Succeeded
+ docker tag <user>/menu-board:1 index.docker.io/<user>/menu-board:1
+ docker push index.docker.io/<user>/menu-board:1
1: digest: sha256:02dd2bbc2049... size: 2196
+ docker push index.docker.io/<user>/menu-board:latest
Finished: SUCCESS
```

Two lines starting `WARNING! Using --password via the CLI is insecure` and `WARNING! Your password will be stored unencrypted in .../menu-board@tmp/...` are printed by Docker for every login from Jenkins. They are normal here: the console still shows only `********`, and Jenkins deletes that temporary folder when the stage ends.

`HTTP 200` is the smoke test passing. `********` is your token, masked. On the instructor's Docker Hub run (November 8) build #1 took 20 seconds and build #2 15 seconds; your first build can take a few minutes while Docker downloads NGINX. Take **Screenshot 1**: the Console Output with `HTTP 200`, the masked login and `Finished: SUCCESS`.

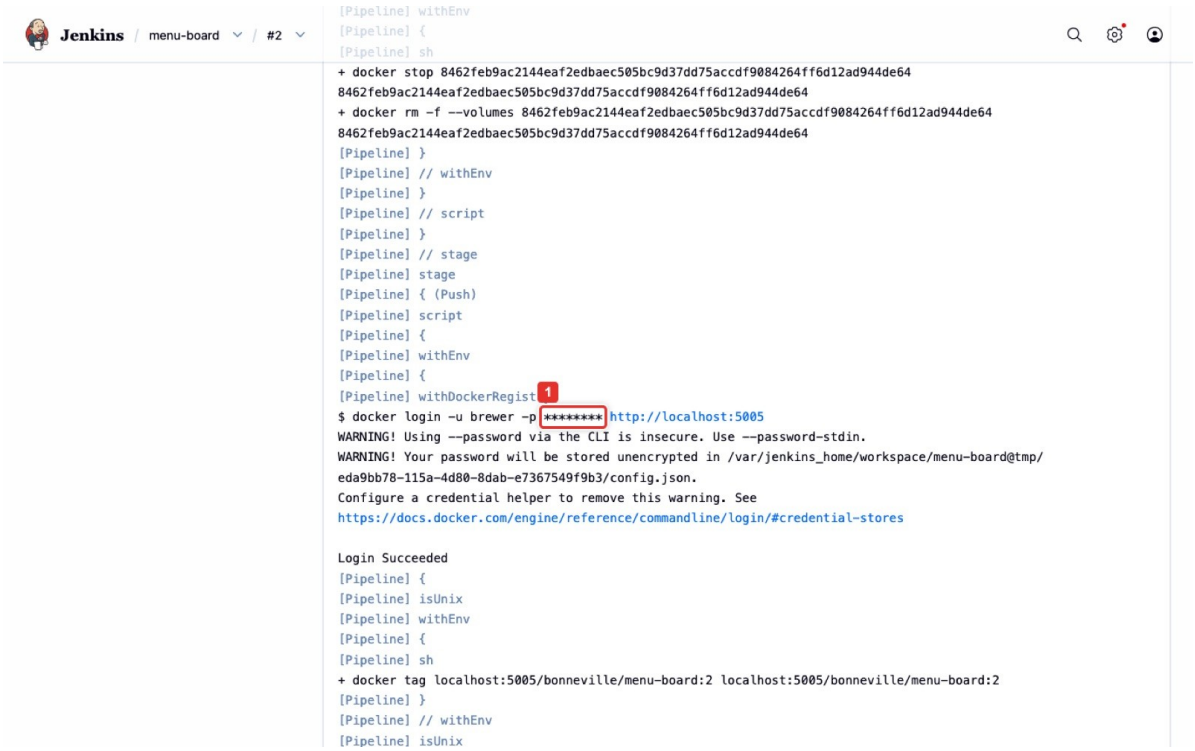


Figure 5. Console Output: docker login with the password printed as `********`.

If not

Couldn't find any revision to build: the branch is wrong. Configure, Branch Specifier `*/main`, Save, Build Now.

permission denied while trying to connect to the Docker daemon socket: run `docker exec -u root jenkins chmod 666 /var/run/docker.sock`, then Build Now.

Smoke test: curl: (7) Failed to connect: Jenkins was started without `--network brewlab -e SMOKE_NET=brewlab`. `docker rm -f jenkins` and redo Part B Step 3 to Step 5 (the credential must be added again).

unauthorized: incorrect username or password at Push: the credential holds your password instead of a token, or a typo. Manage Jenkins > Credentials > dockerhub-credentials > Update, paste the token again.

denied: requested access to the resource is denied at Push: line 4 of the Jenkinsfile does not match your Docker Hub username exactly.

Could not find credentials entry with ID 'dockerhub-credentials': the ID is spelled differently. Fix the ID, not the Jenkinsfile.

Part D. Prove it, change the menu, build again (Registry owner and Quality lead, 10 min)

Step 1. Check the secret is not in the log (Quality lead)

1. Stay on build #1's Console Output.
2. Press Ctrl+F (Mac: Cmd+F) and paste your token into the search box.
3. The browser must find **0 matches**. Then search for ********: you find it on the docker login line.

> **Checkpoint**
>
> If the token can be found anywhere in the log, the job fails the client's requirement. The quality lead writes one line for the submission: "I searched the console output of builds 1 and 2 for the token and found no matches."

Step 2. Check the tags on Docker Hub (Registry owner)

1. Open https://hub.docker.com, then **Repositories** > **menu-board** > **Tags**.
2. You see tags `1` and `latest`. Take **Screenshot 2**.

Docker Hub created the repository for you on the first push. Free accounts make it public, which is fine for a menu.

Step 3. Add a beer and build again (Pipeline author)

1. On GitHub, open `index.html` and click the pencil icon.
2. Just before `</ul>`, add a beer, for example `<li>Timp Trail IPA 6.5%</li>`.
3. Click **Commit changes** twice.
4. In Jenkins, open `menu-board` and click **Build Now**. Build **#2** runs.

> **You should see**
>
> Build #2: `HTTP 200`, the masked login, pushes of `:2` and `:latest`, `Finished: SUCCESS`.
>
> Docker Hub Tags: `2`, `latest` and `1`. Tag `latest` now has the same digest and push time as `2`: it moved. Take **Screenshot 3**.

Step 4. Run the image like a taproom screen would

▸ Run in: Terminal (Mac) or PowerShell (Windows) on your laptop. Any folder. Then open http://localhost:8081 in your web browser.

```
docker run --rm -p 8081:80 <user>/menu-board:2
```

Open http://localhost:8081: the menu shows your new beer. Port 8081 because Jenkins already uses 8080. Stop it with Ctrl+C in the terminal. Then run tag 1 the same way: the new beer is gone. That is a rollback.

> **Verify (your browser)**
>
> http://localhost:8081 with tag **2** lists three beers, the last one **Timp Trail IPA 6.5%** (or the beer you added).
>
> With tag **1** it lists only **Wasatch Wheat 5.2%** and **Antelope Island Amber 5.8%**.
>
> Mac users can also check in Terminal: `curl -s http://localhost:8081 | grep -o "<li>[^<]*</li>"`

Checkpoint: a taproom says the board broke after today's build. Which tag do they go back to?

The previous number, tag `1`. Every build left a numbered tag, so you always know exactly what to go back to. `latest` is a pointer that moves; numbers never move.

Part E. Test connectivity end to end (10 min)

Every hop in this lab is a network connection. Check each one and compare with the instructor's November 8 result.

Hop	How to check	Instructor result (Nov 8)
Your browser to Jenkins	Open http://localhost:8080	Sign-in page, HTTP 200
Jenkins to GitHub	Console: Checking out Revision ...	Checkout of main OK
Jenkins to the smoke-test container	Console: Smoke test stage	HTTP 200
Jenkins (Docker) to Docker Hub	Console: Login Succeeded, two pushes	Login with -p ********, two tags
Secret never in the log	Ctrl+F your token in Console Output	0 matches (checked on builds 1 and 2 with a real Docker Hub token)
Registry: latest moved	Docker Hub Tags: latest and newest number	Same digest
Taproom screen to the image	docker run -p 8081:80 <user>/menu-board:2	Menu with the new beer, HTTP 200

> **Stuck? Ask an AI assistant the right way**
>
> Copy the failing stage's lines from Console Output into GitHub Copilot Chat, ChatGPT/Codex or Gemini and ask what caused it. **Never** paste your token. Check any fix it suggests against the "If not" boxes in this guide before you apply it.

Part F. Submit, then clean up (10 min)

What to submit in Canvas 3.3

- ☐ **Screenshot 1:** build #1 Console Output showing HTTP 200, the masked docker login line and Finished: SUCCESS.
- ☐ **Screenshot 2:** Docker Hub Tags showing 1 and latest.
- ☐ **Screenshot 3:** Docker Hub Tags showing 1, 2 and latest after the second build.
- ☐ The Jenkinsfile (your copy, with your username on line 4).
- ☐ A 2 to 3 sentence summary that names who did what.
- ☐ The quality lead's one-line confirmation that the token appears nowhere in the logs.

Clean up

▸ **Run in: Terminal (Mac) or PowerShell (Windows) on your laptop. Any folder.**

```
docker rm -f jenkins
docker network rm brewlab
docker rmi it360-jenkins <user>/menu-board:1 <user>/menu-board:2 <user>/menu-board:latest
docker ps -a
```

You should see

docker ps -a no longer lists jenkins. Docker Hub's free tier costs nothing, so you can keep the repository. If you used a cloud VM, delete it.

If you think your token was ever exposed (pasted somewhere, visible in a screenshot), delete it in Docker Hub > Personal access tokens and create a new one. That is exactly what a security team would do.

Plan B: no Docker Hub account or GitHub access

On a Mac (or Windows with WSL), the lab folder has a script that runs the whole pipeline against a private registry container on your laptop, with no accounts. It creates Jenkins, the credential, the job, two builds, and checks the logs for the password:

▸ **Run in: Terminal on a Mac, or WSL Ubuntu on Windows (not PowerShell), inside the** jenkins-docker-pipeline **folder.**

```
bash scripts/deploy-lab3-2.sh
bash scripts/rollback-lab3-2.sh -y
```

Use it to see a working pipeline, then do the lab with Docker Hub for your submission. Ask the instructor before submitting a Plan B run.

Troubleshooting at a glance

You see	Do this
docker: command not found or Cannot connect to the Docker daemon	Start Docker Desktop and wait until it says Running.
permission denied ... docker.sock in a build	docker exec -u root jenkins chmod 666 /var/run/docker.sock
Smoke test: connection refused	Restart Jenkins with --network brewlab -e SMOKE_NET=brewlab (Part B Step 3).
unauthorized at Push	Put the access token, not your password, in the credential.

You see	Do this
denied: requested access to the resource is denied	Jenkinsfile line 4 must be <your Docker Hub user>/menu-board.
Couldn't find any revision to build	Branch Specifier */main.
Port 8080 or 8081 already in use	Use another host port: -p 8090:8080 or -p 8082:80.
Jenkins gone after a laptop restart	docker start jenkins, wait a minute, sign in again.

Words to know

Word	Meaning in this lab
Image	A packaged app (here: NGINX plus the menu page) that runs the same on any machine.
Container	A running copy of an image.
Registry	Where images are stored and shared. Docker Hub is a registry.
Tag	A label on an image version: 1, 2, latest.
Pipeline	The automated steps from code to a pushed image, written in the Jenkinsfile.
Credential	A secret stored in Jenkins and used by ID, never shown.
Smoke test	A quick check that the new image starts and answers before it is pushed.